

5. INTRODUCCION A LAS REDES NEURONALES ARTIFICIALES

José J. Martínez P.

jmcursosingenieria111@gmail.com

Septiembre 2026

5.1 Introducción

Una de las fuentes de datos más importantes de los seres vivos son los datos provenientes del ambiente. Los filtran nuestros ojos, nuestros oídos y también otros sentidos. La principal fuente de supervivencia de las especies proviene de estos datos que internamente analiza cada animal en su cerebro, buscando patrones para determinar si hay presas, calculando el esfuerzo para atraparlas, o si son presas para eludir los predadores. De esa interacción permanente con el ambiente realmente se hace Machine Learning, pues son muchos los datos que llegan permanentemente y se van almacenando, a partir de los cuales se logra el conocimiento. En la figura 5.1, el pájaro observa la mariposa y su modelo mental reconoce el tipo de mariposa, para tomar una decisión.



Figura 5.1. Modelo mental para predecir.

El modelo mental elimina detalles no pertinentes y enfatiza en los detalles de interés. En este caso el pájaro, filtra de la cantidad de datos que ofrece el ambiente los importantes para su toma de decisiones, (los features) y con base en su visión y otros sentidos, encuentra patrones y actúa en consecuencia. Cuando el agente no detecta apropiadamente el patrón, pierde esfuerzos y debe realizar cambios en su estructura mental para adaptar el modelo que le permita una mejor anticipación de las consecuencias, cuando se encuentre nuevamente con ese patrón o uno similar. Los agentes que fallan pierden recursos que generalmente favorecen a los exitosos.

Ese cambio en su modelo mental es lo que llamamos aprendizaje a partir de la experiencia. Si se examinan problemas, cuya descripción algorítmica es muy compleja, como identificar en fracciones de segundo una persona entre una multitud, sus soluciones tienen una característica común: la experiencia. Los animales superiores, como el hombre, son capaces de resolver estas situaciones acudiendo a la experiencia acumulada.

Así, de manera muy simplista, es razonable pensar que una forma de solucionar este tipo de problemas es construir sistemas que sean capaces de simular esta propiedad. La observación anterior fue considerada por los científicos de ciencias de la computación en lo que se denomina Neurocomputación cuyo objeto de estudio son las Redes Neuronales Artificiales, RNAs. Las RNAs no son más que un modelo artificial y simplificado del cerebro de los animales. Una red neuronal artificial es un sistema para el procesamiento de datos, cuya unidad básica de procesamiento está inspirada en la neurona.

En el artículo “A Logical Calculus of the Ideas Inmanent in Nervious Activity” de MacCulloch y Pitts (1943) se describe el primer modelo de redes neuronales. Pocos años después Frank Rosemblat publicó el concepto de perceptrón. La neurona artificial o perceptrón es el bloque de construcción y unidad básica de proceso de una red neuronal artificial. Marvin Minsky, junto con Seymour Papert, escribieron el libro “Perceptrones”, donde se establece cómo una sola neurona es incapaz de implementar una función como XOR, y llegaron a que las grandes redes también tienen limitaciones similares. Este concepto limitó el desarrollo de las redes neuronales artificiales por aproximadamente diez años. Luego se retomó el tema y tuvo un primer florecimiento a finales de los 80s y comienzos de los 90s. A finales de los 90s disminuyo nuevamente el interés, pues se consideró que solo resolvían problemas de juguete. En 2005, G. Hinton, premio Nobel de Física 2024, con una red neuronal profunda pudo reconocer dígitos escritos a mano con un 98% de precisión. Los avances han continuado y en 2018, se desarrolló el Transformer, una arquitectura de red, que permitió producir las herramientas de IA generativa, lo que dio la oportunidad a todos los seres humanos de entrar en el mundo de la IA. De manera que las redes neuronales son la columna vertebral de todos estos nuevos desarrollos.

5.2 Modelos de Neuronas

5.2.1 Neurona Natural

La neurona natural, de manera muy simplificada para nuestro interés, consta de un núcleo, un cuerpo denominado soma, al cual están conectadas unas fibras nerviosas llamadas dendritas, fibras terminales de entrada, y un axón, que es una fibra delgada y larga que se divide en su extremo en una serie de fibras terminales de salida.

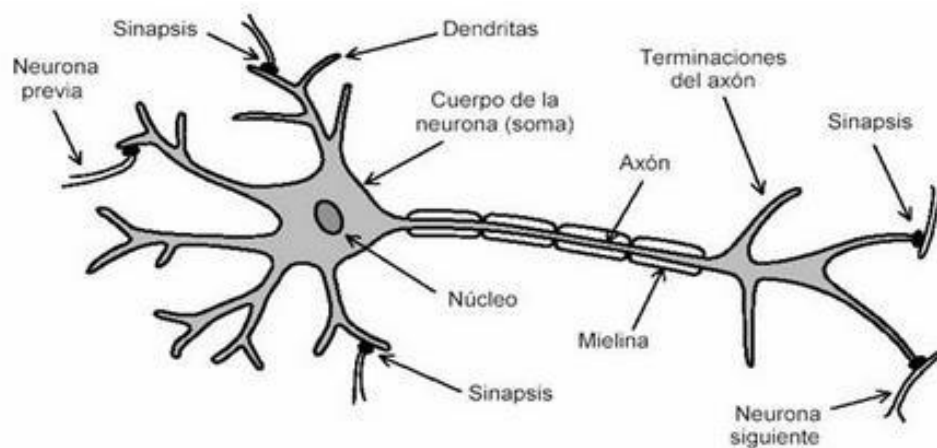


Figura 5.2. Esquema de una neurona natural.

La neurona recibe señales de entrada, provenientes de otras células nerviosas, por las dendritas de la neurona. Las neuronas reciben impulsos de los axones de otras neuronas a través de una conexión llamada sinapsis. La sinapsis (del griego “sin”, que significa “con” y “haptenia”, que significa “con firmeza”) es una conexión más fuerte o más débil dependiendo de la importancia de la señal que llega de la otra neurona. Las señales a través de las cuales se permite la captación y el proceso de control y coordinación de los órganos son de tipo eléctrico. El axón es el encargado de conducir el impulso nervioso desde el soma de una neurona hacia otras células nerviosas.

Las neuronas transmiten su señal eléctrica a consecuencia de un cambio transitorio en la permeabilidad de la membrana plasmática. Cuando el potencial de una membrana, es decir, la suma de todas las señales de entrada se despolariza más allá de un umbral (de 65mV a 55mV), la célula dispara un potencial de acción que se envía a través del axón, como señal de salida de la neurona. El ser humano tiene en promedio 8.6^{10} neuronas y en promedio cada neurona tiene 6.000 conexiones sinápticas, conformando una red con 5.2^{14} enlaces, algo realmente impresionante.

5.2.2 NEURONA ARTIFICIAL

Es un modelo matemático, planteado en términos vectoriales que recibe una serie de señales (valores) X de entrada, que representa el conjunto de dendritas, ponderadas por un peso W , que representa el conjunto de sinapsis, y produce una señal de salida Z , que representa el potencial de salida del axón. La variable representa el umbral θ o el mínimo potencial a partir del cual se dispara el axón. Figura 5.3.

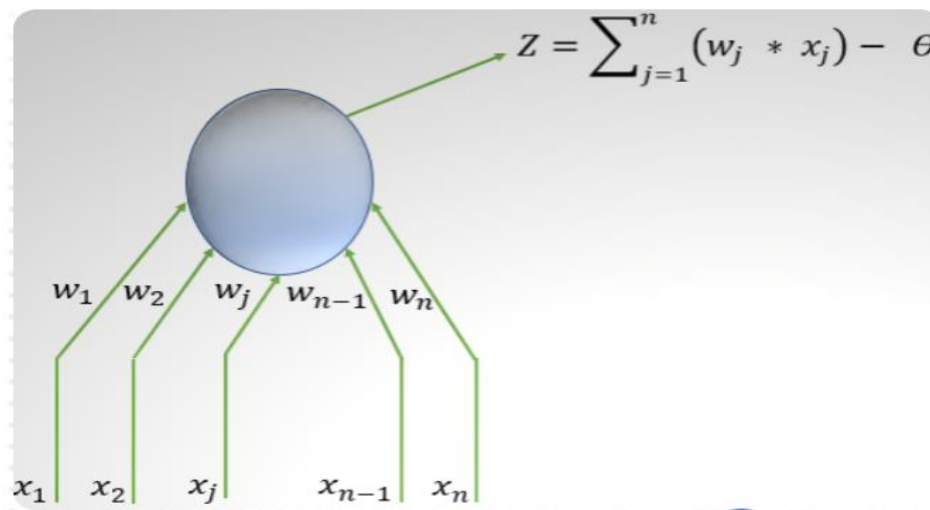


Figura 5.3 Neurona Artificial, modelo abstracto de una neurona natural

Donde

$$Z = -\theta + (w_1 x_1 + w_2 x_2 + \dots, w_j x_j + \dots, +w_n x_n) = \sum_{j=1}^n w_j x_j - \theta,$$

Para propósitos de notación, hacemos $\theta = -b$ (bias). Esta notación es muy importante ya que nos permite trabajar todo el desarrollo matemático en forma vectorial. Entonces tenemos que:

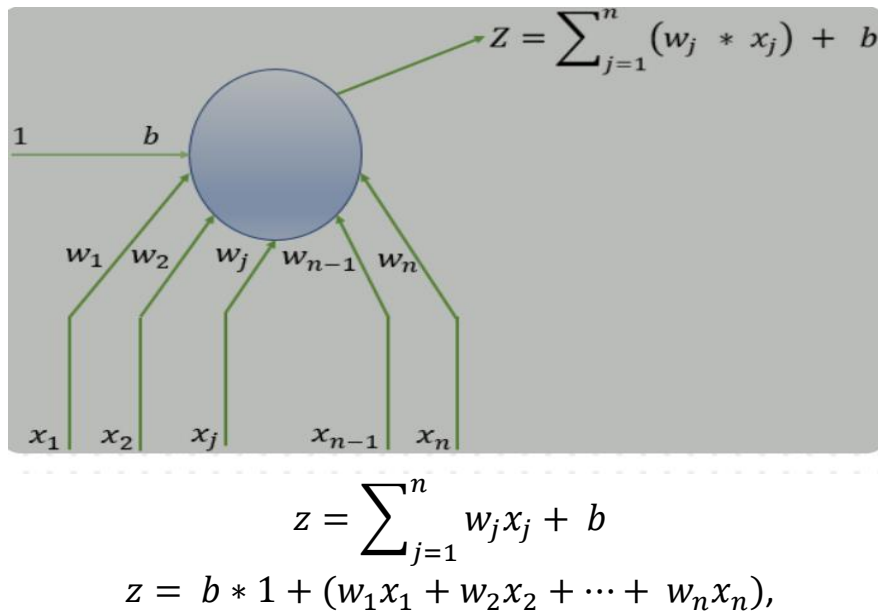


Figura 5.4 Neurona Artificial, modelo abstracto de una neurona natural, con sesgo b .

En términos de aprendizaje de máquina, vamos a denominar el vector X , como el vector de rasgos o **características** (features) y el vector W como el vector de pesos.

$$W = \begin{bmatrix} b \\ w_1 \\ \vdots \\ w_n \end{bmatrix} \quad X = \begin{bmatrix} 1 \\ x_1 \\ \vdots \\ x_n \end{bmatrix}$$

Dado, que la respuesta del axón es una especie de onda, para acercarnos a esta respuesta, definimos la función de activación $g(Z)$, que es la respuesta que realmente nos interesa. Si se observa la ecuación (2), es la ecuación de una recta en un espacio n -dimensional. La función de activación es la que introduce complejidad a la red.

Veamos la función de activación de la figura 5.5, es la función paso unitario, que es una combinación lineal de los valores de entrada X y los pesos W . Donde Z si es mayor que un umbral determinado, toma el valor 1, sino toma el valor 0.

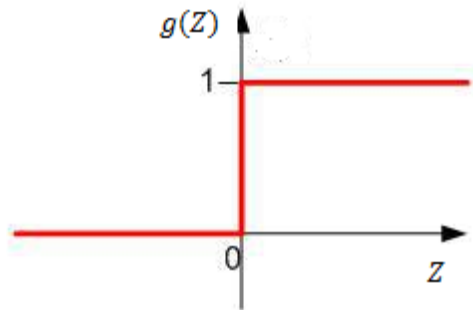


Figura 5.5 Función paso unitario

Sin embargo, esta función tiene varios defectos, en primer lugar, en la naturaleza nunca se presentan cambios en tiempo cero y además es una función que no es derivable, algo bien importante para el algoritmo de aprendizaje. De manera que se utilizan varias funciones de activación que son derivables, la más común es la función de activación sigmoide, figura 5.6.

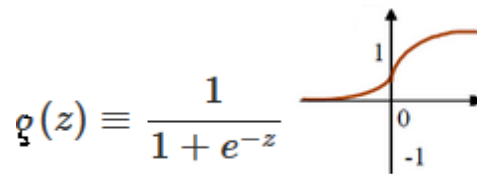


Figura 5.6 Función sigmoide

La función de activación sigmoide tiene la ventaja adicional que: $\sigma' = \sigma(1 - \sigma)$. Es importante recordar que la introducción de la función de activación le da la complejidad a la red neuronal, sin la función de activación, las redes neuronales son simplemente combinaciones lineales.

Aunque la función de activación más usada y más popular ha sido la función sigmoide, la función de activación ReLU (Rectified Linear Activation), se usa en RNAs profundas, porque es más eficiente en el proceso de entrenamiento

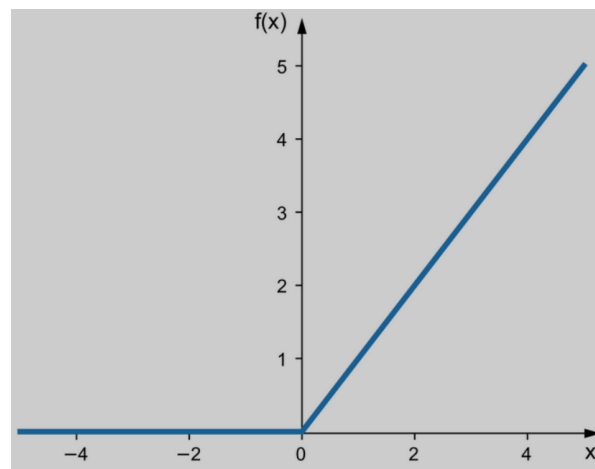


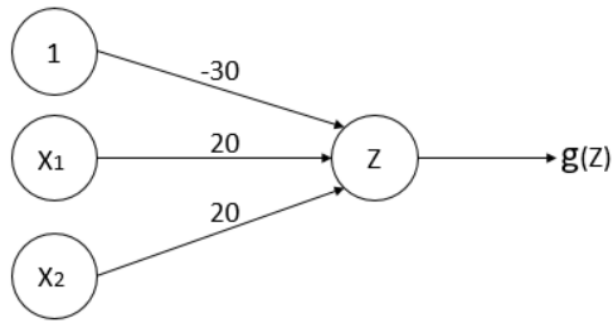
Figura 5.7 Función ReLu

5.3 Ejemplos de clasificación.

Usando la neurona artificial, con la función de activación sigmoide.

Veamos la compuerta AND, figura 5.7.

Con los pesos 30, 20 y 20, se calcula la salida $g(Z)$,

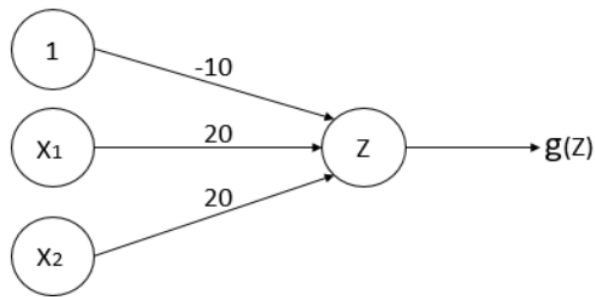


X1	X2	b	w1	w2	Z	g(Z)
0	0	-30	20	20	-30	~ 0
0	1	-30	20	20	-10	~ 0
1	0	-30	20	20	-10	~ 0
1	1	-30	20	20	10	~ 1

Figura 5.7 Red para clasificar una compuerta AND, topología, tabla con vector de pesos.

Ahora la compuerta OR, figura 5.8.

Ahora con los pesos -10, 20 y 20, se calcula $g(Z)$,



X1	X2	b	w1	w2	Z	g(Z)
0	0	-10	20	20	-10	~ 0
0	1	-10	20	20	10	~ 1
1	0	-10	20	20	10	~ 1
1	1	-10	20	20	30	~ 1

Figura 5.8 Red para clasificar una compuerta OR, topología, tabla con vector de pesos.

Como se puede observar ambas redes tienen el mismo número de entradas, solo cambia el peso en la primera entrada, aunque el primero clasifica una compuerta AND, el segundo clasifica una

compuerta OR. El cambio de peso en el vector de sesgo, cambia la función de la red neuronal. ¡Los pesos definen el objetivo de la red neuronal!

Veamos ahora la compuerta NOT, figura 5.9. $Z = (10 - 20x_1)$ y luego se calcula $g(Z)$

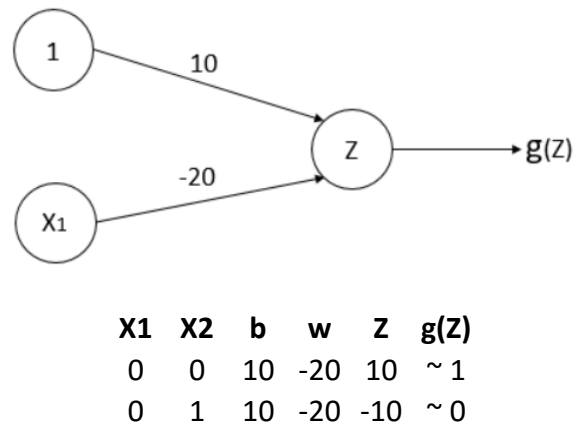


Figura 5.9 Red para clasificar una compuerta NOT, topología, tabla con vector de pesos

Aquí tenemos una clasificación binaria, $Y = [1, 0]$

Ahora, con base en las conformaciones anteriores vamos a armar la red para la clasificación de una compuerta NAND.

$Z^1 = (-30 + 20x_1 + 20x_2)$, luego se calcula $A^1 = g(Z^1)$; luego $Z^2 = (10a_1 - 20a_2)$ y $A^2 = g(Z^2)$;

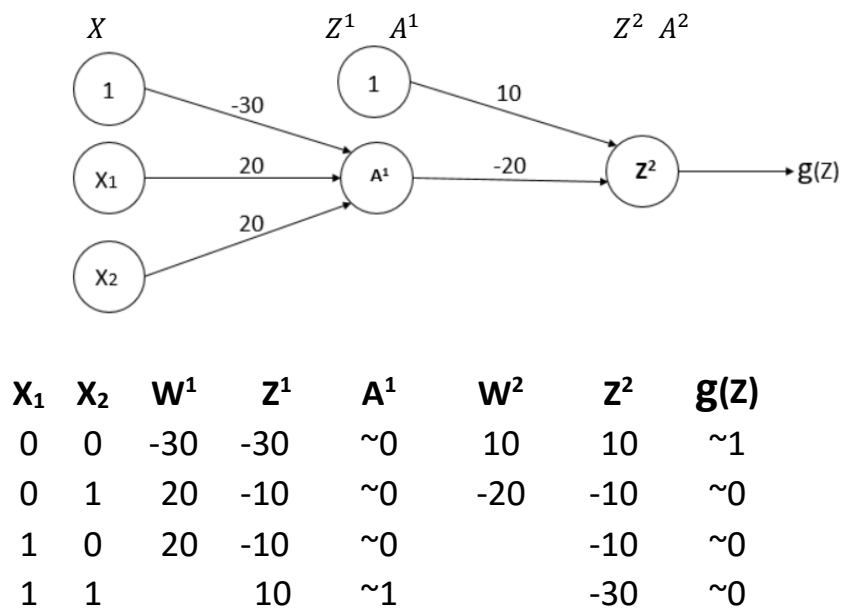


Figura 5.10 Red para clasificar una compuerta NAND, topología y tabla con matriz de pesos

Ahora tenemos dos capas. En la primera capa obtenemos un valor que hemos llamado Z^1 que se calcula con el vector de pesos W^1 y luego aplicando la función sigmoide obtenemos como A^1 resultado intermedio. En la capa 2, obtenemos un valor que hemos llamado Z^2 que se calcula con el vector de pesos W^2 y luego aplicando la función sigmoide obtenemos el resultado $g(Z^2)$ que corresponde a la compuerta NAND. Como podemos observar ahora tenemos una matriz de pesos W .

Podemos hacer lo mismo para obtener la compuerta NOR, uniendo las conformaciones OR y NOT como se hizo en el caso anterior. Pero también podemos unir las redes de clasificación de las compuertas AND y NOR, que nos da la siguiente topología, figura 5.1

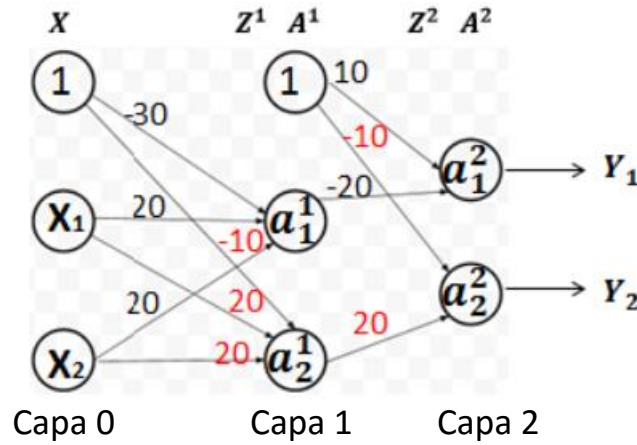


Figura 5.11. Red para clasificar las compuertas AND y OR.

$z_1^1 = (-30 + 20x_1 + 20x_2)$ y $z_2^1 = (-10 + 20x_1 + 20x_2)$; calculamos $a_1^1 = g(z_1^1)$ y $a_2^1 = g(z_2^1)$, que son la salida de la capa 1. Para la capa 2, calculamos $z_1^2 = (10 - 20a_1^1)$ y $z_2^2 = (10 + 20a_2^1)$ y finalmente obtenemos $y_1 = a_1^2 = g(z_1^2)$ y $y_2 = a_2^2 = g(z_2^2)$

Los superíndices nos indican la capa, así, el vector $Z^1 = \begin{bmatrix} z_1^1 \\ z_2^1 \end{bmatrix}$, el vector $A^1 = \begin{bmatrix} a_1^1 \\ a_2^1 \end{bmatrix}$, que son las salidas de la capa 1. La matriz de pesos de la capa 1 es $W^1 = [w_1^1 \ w_2^1]$, $B^1 = [b_1^1 \ b_2^1]$ y $W^2 = [w_1^2 \ w_2^2]$, $B^2 = [b_1^2 \ b_2^2]$:

$$W^1 = \begin{bmatrix} w_1^1 & w_2^1 \\ 20 & 20 \\ 20 & 20 \end{bmatrix} \quad B^1 = \begin{bmatrix} -30 & -10 \end{bmatrix} \quad W^2 = \begin{bmatrix} w_1^2 & w_2^2 \\ -20 & -20 \end{bmatrix} \quad B^2 = \begin{bmatrix} 10 & 10 \end{bmatrix}$$

En este caso tenemos una multi-clasificación, es decir $g(Z^2)$ es un vector. En términos de clasificación. Si k es el número de clases, $k = 2$, en este caso tenemos dos clases NAND y NOR.

$$Y = \begin{array}{|c|c|} \hline Y1 & Y2 \\ \hline 1 & 0 \\ \hline 0 & 1 \\ \hline \end{array}$$

Si por ejemplo estamos clasificando números arábigos, tendríamos $k = 10$.

Las redes neuronales artificiales están diseñadas para procesar información y aprender patrones complejos a partir de los datos. Sus funciones principales son la clasificación y la predicción.

Clasificación

Se puede definir de manera general la clasificación como el agrupamiento de cosas que comparten ciertas características y cualidades. Por ejemplo, clasificar formas geométricas, números escritos a mano, o simplemente huevos. En IA, la clasificación es el problema de identificar a que conjunto de categorías pertenece una nueva observación, con base en un proceso computacional en el que se utiliza un modelo algorítmico y en el que se ha tenido un conjunto de datos de entrenamiento que contiene observaciones cuya membresía de categorías se conoce.

Predicción

Con base en datos históricos, datasets, las redes neuronales, pueden predecir o estimar futuros valores numéricos en un determinado horizonte de tiempo. Es importante ser conscientes de que siempre estas predicciones tienen una incertidumbre inherente.

5.4 Aprendizaje Supervisado

Uno de los procesos más importantes en RNA es el proceso de aprendizaje. El aprendizaje puede hacerse de manera supervisada o no supervisada. En el capítulo siguiente volveremos sobre este tema. En el aprendizaje o entrenamiento supervisado, se proveen tanto las entradas, las características, como las salidas, o r tulos, correspondientes a esas entradas. Un atributo, o característica, es un aspecto espec fico de un objeto, o del r tulo. En ML se supone que el conjunto de caracter sticas describe un objeto.

La red entonces procesa las entradas y compara sus salidas resultantes contra las salidas deseadas o r tulos. En la mayor a de los casos no corresponden y se debe calcular su diferencia o error. Entonces ahora se busca ajustar los pesos que controlan la red para disminuir el error, hasta que de acuerdo con un determinado criterio se considera que el error es aceptable.

Generalizaci n del aprendizaje

Se busca que con la red neuronal se pueda, por ejemplo, predecir apropiadamente el costo de cualquier inmueble, es decir la red debe generalizar el concepto para cualquier tipo de inmueble. Se pueden presentar dos problemas, que el modelo sea demasiado general o que sea demasiado espec fico. Esto se debe tener en cuenta al entrenar la red.

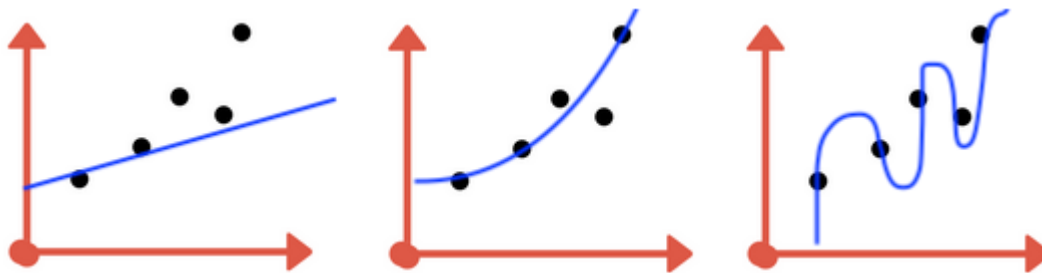


Figura 5.12. En la figura de la izquierda se tiene sub-entrenamiento, también conocido como underfitting; en el centro, el entrenamiento adecuado, y en la figura de la derecha se tiene sobre-entrenamiento o overfitting.

Para solucionar este problema, el dataset se divide en tres conjuntos, el conjunto de entrenamiento, generalmente del 80% del dataset; el conjunto de prueba, con el que se busca obtener una estimación del comportamiento de la red en el mundo real; y el conjunto de validación, que nos permite una evaluación de la generalización del modelo.

Propagación

Las redes que hemos visto hasta ahora se conocen como feedforward, cuya característica es que el proceso va desde la capa de entrada hasta capa de salida. No hay retroalimentación hacia atrás. Es la red más simple y utilizada. Entonces, vamos a generalizar una red feed-forward, con su nomenclatura, con base en la figura 5.13.

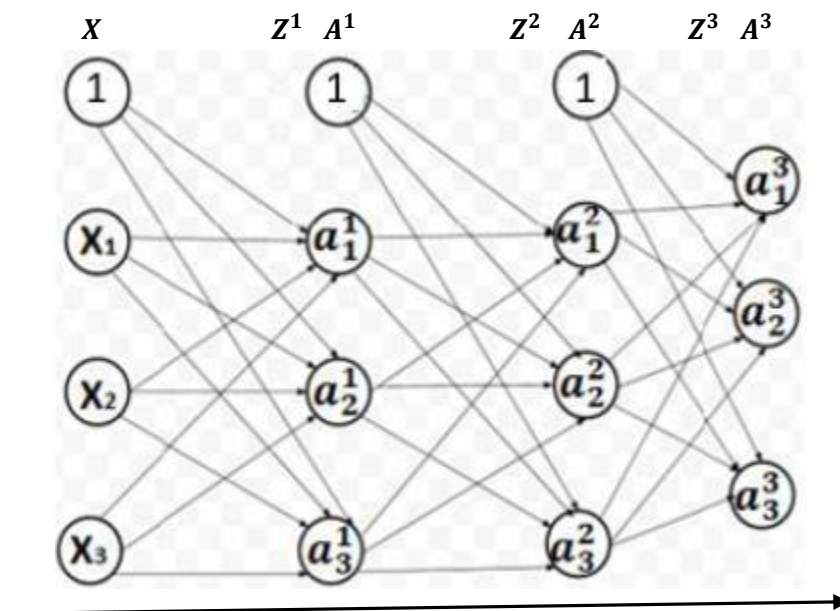


Figura 5.13. Red de tres capas para mostrar la propagación, desde el vector de entrada.

Vamos a describir cómo se propagan los datos en la red desde la entrada.

Capa 1:

$$z_1^1 = b_1^1 + (w_{11}^1 x_1 + w_{21}^1 x_2 + w_{31}^1 x_3), \quad a_1^1 = g(z_1^1)$$

$$z_2^1 = b_2^1 + (w_{12}^1 x_1 + w_{22}^1 x_2 + w_{32}^1 x_3), \quad a_2^1 = g(z_2^1)$$

$$z_3^1 = b_3^1 + (w_{13}^1 x_1 + w_{23}^1 x_2 + w_{33}^1 x_3), \quad a_3^1 = g(z_3^1)$$

con

$$W_T^1 = B^1 + W^1 = \begin{matrix} & b_1^1 & w_{11}^1 & w_{21}^1 & w_{31}^1 \\ b_2^1 & + & w_{12}^1 & w_{22}^1 & w_{32}^1 \\ b_3^1 & & w_{13}^1 & w_{23}^1 & w_{33}^1 \end{matrix}$$

Matriz de pesos de la capa 1

Matricialmente, $Z^{(1)} = B^1 + (W^{(1)T})X$ y $A^{(1)} = g(Z^{(1)})$; para generalizar podemos decir que $A^{(0)} = X$, y así la ecuación matricial para la capa 1 sería $Z^{(1)} = B^1 + (W^{(1)T})A^{(0)}$.

Capa 2:

$$z_1^2 = b_1^2 + (w_{11}^2 a_1^1 + w_{21}^2 a_2^1 + w_{31}^2 a_3^1), \quad a_1^2 = g(z_1^2)$$

$$z_2^2 = b_2^2 + (w_{12}^2 a_1^1 + w_{22}^2 a_2^1 + w_{32}^2 a_3^1), \quad a_2^2 = g(z_2^2)$$

$$z_3^2 = b_3^2 + (w_{13}^2 a_1^1 + w_{23}^2 a_2^1 + w_{33}^2 a_3^1), \quad a_3^2 = g(z_3^2)$$

con

$$W_T^2 = B^2 + W^2 = \begin{matrix} & b_1^2 & w_{11}^2 & w_{21}^2 & w_{31}^2 \\ b_2^2 & + & w_{12}^2 & w_{22}^2 & w_{32}^2 \\ b_3^2 & & w_{13}^2 & w_{23}^2 & w_{33}^2 \end{matrix}$$

Matriz de pesos de la capa 2.

Matricialmente $Z^{(2)} = B^2 + (W^{(2)T})A^{(1)}$

Capa 3:

$$z_1^3 = b_1^3 + (w_{11}^3 a_1^2 + w_{21}^3 a_2^2 + w_{31}^3 a_3^2), \quad a_1^3 = g(z_1^3)$$

$$z_2^3 = b_2^3 + (w_{12}^3 a_1^2 + w_{22}^3 a_2^2 + w_{32}^3 a_3^2), \quad a_2^3 = g(z_2^3)$$

$$z_3^3 = b_3^3 + (w_{13}^3 a_1^2 + w_{23}^3 a_2^2 + w_{33}^3 a_3^2), \quad a_3^3 = g(z_3^3)$$

$$W_T^3 = B^3 + W^3 = \begin{matrix} & b_1^3 & w_{11}^3 & w_{21}^3 & w_{31}^3 \\ b_2^3 & + & w_{12}^3 & w_{22}^3 & w_{32}^3 \\ b_3^3 & & w_{13}^3 & w_{23}^3 & w_{33}^3 \end{matrix}$$

Matriz de pesos de la capa 3

Matricialmente

$Z^{(3)} = B^3 + (W^{(3)T})A^{(2)}$ y $A^{(3)} = g(Z^{(3)})$. Generalizando, para cualquier capa oculta i , se tiene que:

$$Z^{(i)} = B^{(i)} + (W^{(i)T})A^{(i-1)} \text{ y } A^{(i)} = g(Z^{(i)}).$$

Al analizar esta ecuación, encontramos que se pasa de una capa a la otra con la ecuación de la izquierda y el vector de salida $A^{(i)}$ se encuentra aplicando la función de activación a $Z^{(i)}$.

Volviendo a la figura 5.11, como la capa 3 es la última capa de la red, los $A^{(3)}$ son la respuesta de la red que notamos como $H_w(X)$. En otras palabras, el resultado de la propagación del vector X a través de la red, es el vector $H_w(X)$.

Aprendizaje supervisado

Con el fin de encontrar los pesos apropiados para que la red responda apropiadamente a los patrones que queremos, debemos trabajar en el ajuste de los pesos. Para entrenar la red en esos patrones hacemos **aprendizaje supervisado**, proveyendo, para este proceso un conjunto de entrenamiento: $(X_{(1)}, Y_{(1)}), (X_{(2)}, Y_{(2)}), \dots, (X_{(m)}, Y_{(m)})$. Donde los vectores X son las características (features) o patrones de entrada y los vectores Y son los patrones que debe reconocer o las clasificaciones que debe producir la red.

Analicemos un poco el conjunto de entrenamiento, por ejemplo el conjunto de entrenamiento de MNIST que consiste en dígitos escritos a mano de 10.000 ejemplos. Los dígitos son imágenes de un tamaño normalizado de 28 X 28 pixeles, 784 pixeles, en escala de grises, con un rótulo que dice que número es, de manera que los valores de los rótulos son de 0 a 9. De manera que en el par $(X_{(m)}, Y_{(m)})$, el $X_{(m)}$ corresponde a un vector de 784 valores y $Y_{(m)}$ corresponde a un rótulo entre 0 y 9. Así, en la figura 5.15, los valores en el recuadro son los corresponden a los 784 valores de los pixeles el número escrito a mano y la flecha me indica su rótulo

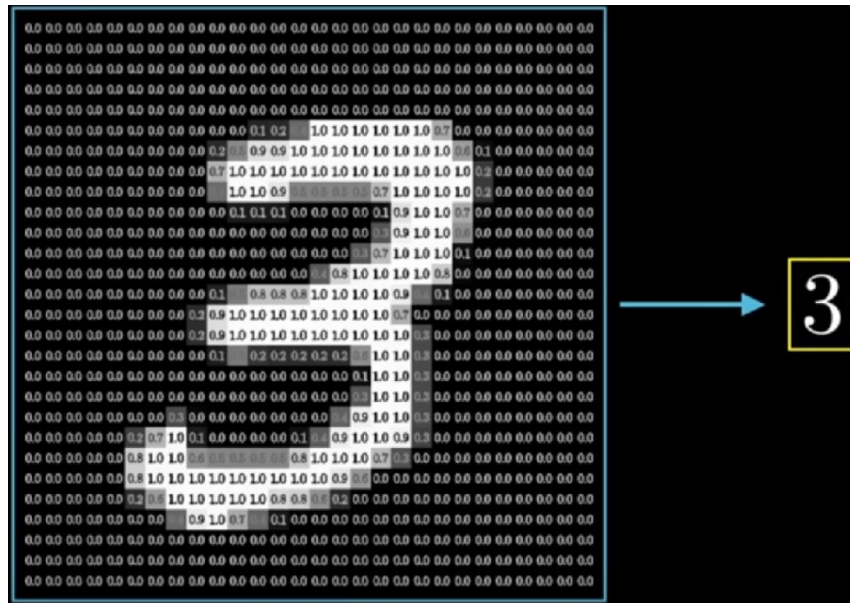


Figura 5.14. El número 3 escrito a mano, normalizado como una imagen de 28 x 28, 784 píxeles, y su rótulo correspondiente, 3.

Se trata entonces de darle como entrada a una red neuronal los 784 píxeles y un rótulo; pero 10.000 veces, para que se obtengan los pesos que permitan clasificar cualquier número escrito a mano, normalizado a 28 X 28 píxeles, ver figura 5.15.

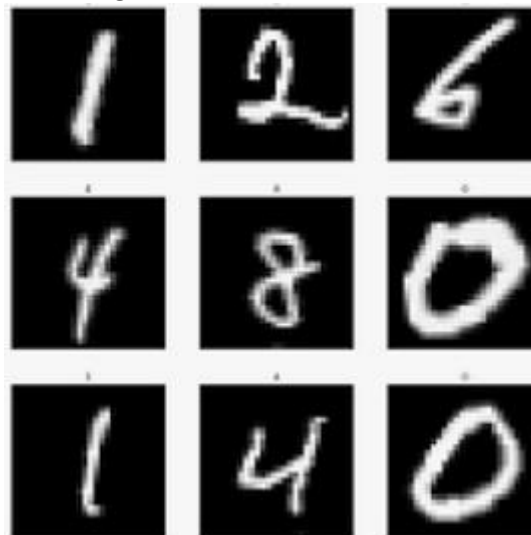


Figura 5.15. En la figura aparecen los números escritos a mano como imágenes de 28 X 28, 784 píxeles, valores de X , aparte vienen los valores de los rótulos 1, 2, 6, 4, 8, 0, 1, 4, 0; que corresponden a los valores de Y , o rótulos.

5.5 Algoritmo de aprendizaje y función de pérdida

La topología de la red depende del número de características (features) del vector de entrada, del número de salidas en la capa de salida, que corresponde a las clasificaciones

que hace la red o del resultado que se espera y del número de capas ocultas, que generalmente es de dos o tres capas. Además, todas las neuronas de la capa i se conectan con todas las neuronas de la capa $i + 1$; grafos completamente conectados

Inicialmente se asignan valores aleatorios entre $[0, 1]$ a los pesos de la red, aunque algunos autores recomiendan valores menores de 0.5. Una vez se hace la propagación a través de la red de los vectores X , con esos valores de pesos, se obtiene el vector de salida Y . La diferencia con el

rotulo de entrenamiento Y_r , es $e = |Y_r - Y|$, es el error de clasificación de la red $e = \begin{bmatrix} e_1 \\ e_2 \\ e_3 \end{bmatrix}$

El proceso de entrenamiento o aprendizaje consiste en hacer que ese error sea menor que un ϵ definido. También se define la función de pérdida como el error promedio E como:

$$E = \frac{1}{2} (e)^2 = \frac{1}{2} (e)^T e$$

La idea es minimizar el error haciendo que se afecten todos los pesos en cada una de las capas de la red. Como solo se conoce el error en la última capa, se comienza minimizando el error en esa capa, ajustando el vector de sesgo y la matriz de peso de esa capa. Y luego se sigue hacia las capas internas, hasta llegar a la capa de entrada. Esto es lo que se conoce como algoritmo de retropropagación o Backpropagation.

Backpropagation.

Tercera capa. Dado que tenemos 3 capas, se minimiza el error con respecto a los pesos en la capa 3, en este caso la capa de salida, ver figura 5.16. Siguiendo la regla de la cadena se tiene:

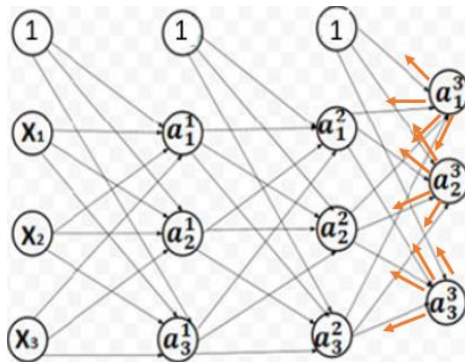


Figura 5.16, Con base en el error promedio E , se ajustan los valores de la matriz de pesos de la última capa.

De manera que se busca encontrar el cambio en el error promedio E con base en el cambio en la matriz de pesos $W_T^3 : \frac{\partial E}{\partial W_T^3}$. Pero lo que conocemos es Z^3 , de manera que hacemos:

$$\frac{\partial E}{\partial W_T^3} = \frac{\partial E}{\partial Z^3} \frac{\partial Z^3}{\partial W_T^3}$$

O También:

$$\frac{\partial E}{\partial W_T^3} = \frac{\partial Y}{\partial Z^3} \frac{\partial e}{\partial Y} \frac{\partial E}{\partial e} \frac{\partial Z^3}{\partial W_T^3}, \text{ recordando que } A^3 = Y = g(Z^3). Y = \sigma(Z^3)$$

Es importante recordar que $\sigma' = \sigma(1 - \sigma)$

$$\text{Así, } \frac{\partial Y_i^3}{\partial Z_j^3} = \begin{cases} 1, & \text{si } i = j \\ 0, & \text{si } i \neq j \end{cases} \quad \text{o} \quad \frac{\partial Y^3}{\partial Z^3} = \begin{bmatrix} \sigma'(Z_1^3) & 0 & 0 \\ 0 & \sigma'(Z_2^3) & 0 \\ 0 & 0 & \sigma'(Z_3^3) \end{bmatrix} \text{ y notando}$$

$$\text{Nos queda que: } \frac{\partial Y}{\partial Z^3} = \text{diag}(\sigma'(Z^3)). \text{ Recordando } e = |Y_r - Y|$$

$$\text{Se tiene, } \frac{\partial e}{\partial Y} = \frac{\partial e_i}{\partial Y_j} = \begin{cases} -1, & \text{si } i = j \\ 0, & \text{si } i \neq j \end{cases} \quad \text{o} \quad \frac{\partial e_i}{\partial Y_j} = - \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = -1$$

1

$$\text{Ahora, } \frac{\partial E}{\partial e} = \frac{\partial}{\partial e} \left(\frac{1}{2} (e)^T e \right) = e. \text{ Recordando } Z^{(3)} = B^{(3)} + (W^{(3)T})A^{(2)}$$

$$\text{Ahora, } \frac{\partial Z^3}{\partial W_T^3} = \frac{\partial}{\partial W_T^3} (B^{(3)} + (W^{(3)T})A^{(2)}) \quad \text{o, } \frac{\partial Z^3}{\partial W_T^3} = 1 + A^{(2)T}$$

Reuniendo todos los términos:

$$\begin{aligned} \frac{\partial E}{\partial W_T^3} &= \frac{\partial Y^3}{\partial Z^3} \frac{\partial e}{\partial Y^3} \frac{\partial E}{\partial e} \frac{\partial Z^3}{\partial W_T^3} \\ \frac{\partial E}{\partial W_T^3} &= \text{diag}(\sigma'(Z^3)) - 1 e (1 + A^{(2)T}) \\ \frac{\partial E}{\partial W_T^3} &= -\text{diag}(\sigma'(Z^3)) e (1 + A^{(2)T}) \end{aligned}$$

Haciendo $\delta^3 = \text{diag}(\sigma'(Z^3)) e$, entonces

$$\frac{\partial E}{\partial W_T^3} = -\delta^3 (1 + A^{(2)T}) \quad \text{o} \quad \frac{\partial E}{\partial W_T^3} = -\delta^3 - \delta^3 A^{(2)T}$$

$$\text{donde } \frac{\partial E}{\partial W_T^3} = \frac{\partial E}{\partial B^3} + \frac{\partial E}{\partial W^3}$$

$$\text{con } \frac{\partial E}{\partial B^3} = -\delta^3 \quad \text{y} \quad \frac{\partial E}{\partial W^3} = -\delta^3 A^{(2)T}$$

De manera que con estos valores ajustamos lo pesos de la última capa.

Segunda capa. Ahora se minimiza el error promedio E con respecto a los pesos de la capa 2, ver figura 5.17. Siguiendo la regla de la cadena se tiene:

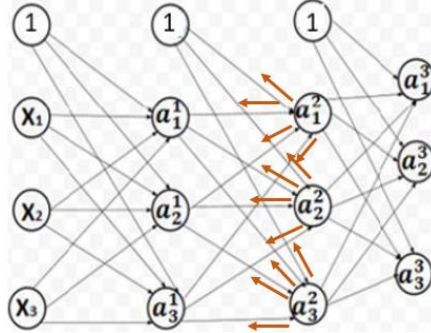


Figura 5.17. Con base en el error promedio E , se ajustan los valores de la matriz de pesos de la segunda capa.

$$\frac{\partial E}{\partial W_T^2} = \frac{\partial E}{\partial Z^2} \frac{\partial Z^2}{\partial W_T^2}$$

$$\frac{\partial E}{\partial W_T^2} = \frac{\partial A^2}{\partial Z^2} \frac{\partial Z^3}{\partial A^2} \frac{\partial E}{\partial Z^3} \frac{\partial Z^2}{\partial W_T^2}$$

$$\text{Ahora, } \frac{\partial A_i^2}{\partial Z_j^2} = \begin{cases} 1, & \text{si } i = j \\ 0, & \text{si } i \neq j \end{cases} \quad \text{o} \quad \frac{\partial A^2}{\partial Z^2} = \begin{bmatrix} \sigma'(Z_1^2) & 0 & 0 \\ 0 & \sigma'(Z_2^2) & 0 \\ 0 & 0 & \sigma'(Z_3^2) \end{bmatrix}$$

$$\frac{\partial A^2}{\partial Z^2} = \text{diag}(\sigma'(Z^2))$$

$$\text{Ahora, } \frac{\partial Z^3}{\partial A^2} = \frac{\partial}{\partial W_T^3} (B^{(3)} + (W^{(3)T})A^{(2)}) \quad \text{o,} \quad \frac{\partial Z^3}{\partial A^2} = W^{(3)T}$$

Ahora, para calcular $\frac{\partial E}{\partial Z^3}$ Teníamos que: $E = \frac{1}{2} (e)^2$ o $E = \frac{1}{2} (Y_r - Y)^2$ o $E = \frac{1}{2} (Y_r - g(Z^{(3)}))^2$ ahora, derivando con respecto a $Z^{(3)}$, se tiene:

$$\frac{\partial E}{\partial Z^3} = (Y_r - g(Z^{(3)})) - \text{diag}(\sigma'(Z^3)), \text{ pero } e = |Y_r - g(Z^{(3)})|, \text{ entonces reacomodando}$$

$$\frac{\partial E}{\partial Z^3} = -\text{diag}(\sigma'(Z^3)) e, \text{ y se había definido } \delta^3 = \text{diag}(\sigma'(Z^3)) e$$

$$\frac{\partial E}{\partial Z^3} = -\delta^3$$

Ahora, $\frac{\partial Z^2}{\partial W_T^2} = \frac{\partial}{\partial W_T^2} (B^{(2)} + W^{(2)T} A^1)$ o, $\frac{\partial Z^2}{\partial W_T^2} = 1 + A^{(1)T}$

Reuniendo los términos

$$\frac{\partial E}{\partial W_T^2} = -\text{diag}(\sigma'(Z^2)) W^{(3)T} \delta^3 (1 + A^{(1)T})$$

Haciendo $\delta^2 = \text{diag}(\sigma'(Z^2)) W^{(3)T} \delta^3$, entonces

$$\frac{\partial E}{\partial W_T^2} = -\delta^2 (1 + A^{(1)T}) \text{ o } \frac{\partial E}{\partial W_T^2} = -\delta^2 - \delta^2 A^{(1)T}$$

Donde $\frac{\partial E}{\partial B^2} = -\delta^2$ y $\frac{\partial E}{\partial W^2} = -\delta^2 A^{(2)T}$

$$\frac{\partial E}{\partial W_T^3} = \frac{\partial E}{\partial B^3} + \frac{\partial E}{\partial W^3}$$

Primera capa. Ahora se minimiza el error promedio E con respecto a los pesos de la capa 1, ver figura 5.18. Siguiendo la regla de la cadena se tiene:

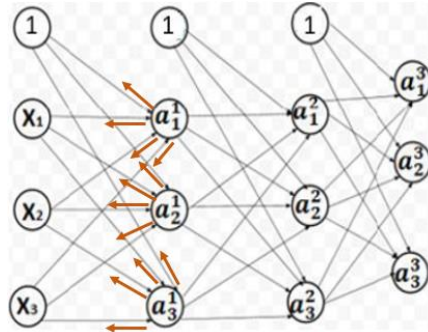


Figura 5.18. Con base en el error promedio E , se ajustan los valores de la matriz de pesos de la primera capa.

$$\frac{\partial E}{\partial W_T^1} = \frac{\partial E}{\partial Z^1} \frac{\partial Z^1}{\partial W_T^1}$$

$$\frac{\partial E}{\partial W_T^1} = \frac{\partial A^1}{\partial Z^1} \frac{\partial Z^2}{\partial A^1} \frac{\partial E}{\partial Z^2} \frac{\partial Z^1}{\partial W_T^1}$$

Ahora, $\frac{\partial A_i^1}{\partial Z_j^1} = \begin{cases} 1, si i = j \\ 0, si i \neq j \end{cases}$ o $\frac{\partial A_i^1}{\partial Z_j^1} = \begin{bmatrix} \sigma'(Z_1^1) & 0 & 0 \\ 0 & \sigma'(Z_1^1) & 0 \\ 0 & 0 & \sigma'(Z_1^1) \end{bmatrix}$

$$\frac{\partial A^1}{\partial Z^1} = \text{diag}(\sigma'(Z^1))$$

Ahora, $\frac{\partial Z^2}{\partial A^1} = \frac{\partial}{\partial A^1} (B^{(2)} + W^{(2)T} A^1)$ o $\frac{\partial Z^2}{\partial A^1} = W^{(2)T}$

Ahora, para calcular $\frac{\partial E}{\partial Z^2}$ Teníamos que: $E = \frac{1}{2} (e)^2$ o $E = \frac{1}{2} (Y - Y_c)^2$ o
 $E = \frac{1}{2} (Y - g(Z^{(3)}))^2$, reemplazando $Z^{(3)}$ por $B^3 + (W^{(3)T})A^{(2)}$ se tiene:
 $E = \frac{1}{2} (Y - g(B^3 + (W^{(3)T})A^{(2)}))^2$, pero $A^{(2)} = g(Z^{(2)})$; reemplazando:
 $E = \frac{1}{2} (Y - g(B^3 + (W^{(3)T})g(Z^{(2)})))^2$ y derivando

$$\frac{\partial E}{\partial Z^2} = (Y - g(Z^{(3)})) (-\text{diag}(\sigma'(Z^3))) W^{(3)T} \text{diag}(\sigma'(Z^2))$$

$$\frac{\partial E}{\partial Z^2} = -\text{diag}(\sigma'(Z^3)) e W^{(3)T} \text{diag}(\sigma'(Z^2)) \text{ pero } \delta^3 = \text{diag}(\sigma'(Z^3)) e$$

$$\frac{\partial E}{\partial Z^2} = \delta^3 W^{(3)T} \text{diag}(\sigma'(Z^2)) \text{ y } \delta^2 = \text{diag}(\sigma'(Z^2)) W^{(3)T} \delta^3$$

$$\frac{\partial E}{\partial Z^2} = \delta^2$$

Ahora, $\frac{\partial Z^2}{\partial A^1} = \frac{\partial}{\partial A^1} (B^{(2)} + W^{(2)T} A^1)$,

$$\frac{\partial Z^1}{\partial W_T^1} = 1 + A^{(0)T}$$

$$\frac{\partial E}{\partial W_T^1} = -\text{diag}(\sigma'(Z^1)) W^{(2)T} \delta^2 (1 + A^{(0)T})$$

Haciendo $\delta^1 = \text{diag}(\sigma'(Z^1)) W^{(2)T} \delta^2$, entonces

$$\frac{\partial E}{\partial W_T^1} = -\delta^1 (1 + A^{(0)T})$$

Donde $\frac{\partial E}{\partial B^1} = -\delta^1$ y $\frac{\partial E}{\partial W^1} = -\delta^1 A^{(1)T}$

En este caso la operación + nos indica se debe considerar una operación con dos componentes, uno que es la actualización de los pesos que representan el umbral de manera separada y el otro que corresponde a la actualización de la matriz de pesos.

Actualización de los pesos

Los pesos se actualizan gradualmente usando el descenso del gradiente. Con η como la tasa de aprendizaje.

Generalizando, para cualquier capa interna:

$$\delta^k = \text{diag}(\sigma'(Z^k)) W^{(k+1)T} \delta^{k+1}$$

$$\Delta W^{(K+1)} = -\eta \delta^k - \eta \delta^k A^{(k-1)T}$$

$$W^{(K+1)} = W^{(K)} + \Delta W^{(K+1)}$$

En las fórmulas anteriores, la variable η se conoce como tasa de aprendizaje. Es un valor que se toma entre 0 y 1. Si por ejemplo se toma $\eta = 0.2$, quiere decir que solamente se tiene en cuenta el 20% del error para la actualización de los pesos, se considera como un hiperparámetro de configuración. Los pesos también se consideran hiperparámetros.

Resumiendo, en el aprendizaje por Backpropagation se calcula el error en la salida. Luego se pasa el error hacia atrás y se pondera a lo largo de cada capa, permitiendo calcular los cambios que se deben realizar en la matriz W de pesos y el vector B de sesgos, en cada capa. Cuando se llega a una unidad, se propaga, se multiplica el error propagado hacia atrás por la derivada de la unidad. Es un algoritmo recursivo.

El algoritmo de retropropagación fue descrito inicialmente por P. Werbos en 1974, pero G. Hinton y su grupo, mostraron la eficiencia en el entrenamiento de redes multicapa en los 80s.

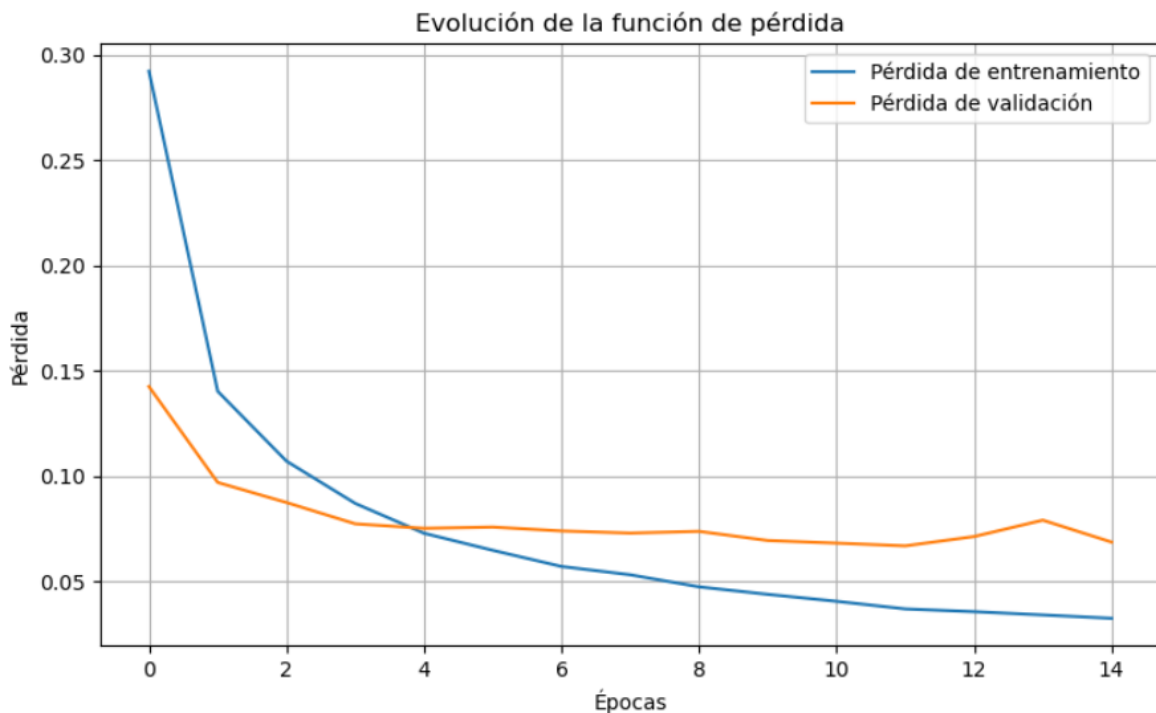


Figura 5.17. Cómo va evolucionando la función de pérdida, con 15 épocas.

El dataset en el aprendizaje supervisado, incluye la respuesta correcta Y_r , o valores rotulados, para cada ejemplo X , el par (X, Y_r) .

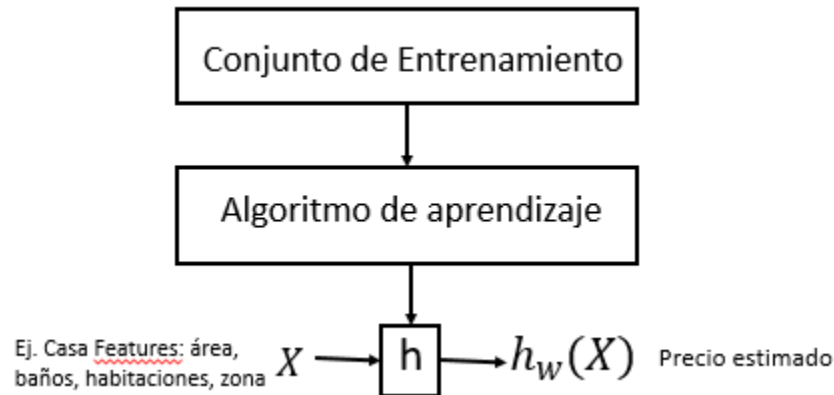
Notación

m = número de ejemplos de entrenamiento.

X = variable de entrada, es un vector con todas las características (features) del ejemplo.

n = número de características (features) de la variable de entrada.

Y = variable de salida, valores objetivo (target), o valores rotulados.



En las diferentes heurísticas de ML, se define una función de costo J que es la diferencia entre la hipótesis que produce un modelo $h_w(X)$ y el valor real Y en el conjunto de aprendizaje:

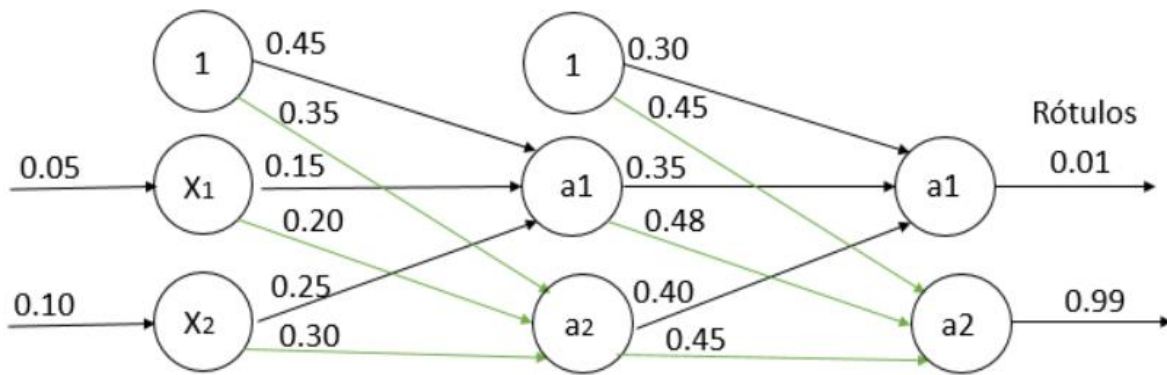
$$J(W) = \frac{1}{2m} \left(\sum_{i=1}^m (h_w(X) - Y)^2 \right)$$

Donde $\frac{1}{2} (h_w(X) - Y)^2$, es el error cuadrático que se ha trabajado, para el conjunto de aprendizaje. Cuando se tiene en el conjunto de aprendizaje m ejemplos, lo que nos dice la ecuación es que se toma el promedio de todos los errores al entrenar la red con todos los m ejemplos del conjunto de entrenamiento.

La función de costo también se conoce como función de pérdida y tal vez es el nombre que más se utiliza. Para ML hay varias funciones de pérdida.

5.6 Ejemplos

Ejemplo 1. Suponga que tiene la siguiente red neuronal:



Vamos a ver el programa que se está en NoteBook de Jupyter.

```
1 import numpy as np
```

```
1 nn_0 = 2
2 nn_1 = 2
3 nn_2 = 2
4
5 b1 = np.array([[0.45], [0.35]])
6 W1 = np.array([[0.15, 0.20], [0.25, 0.30]]) # Inicialización de los sesgos y los pesos
7 b2 = np.array([[0.30], [0.45]])
8 W2 = np.array([[0.35, 0.48], [0.40, 0.45]])
9
10 Z1 = np.zeros((nn_1,1))
11 A1 = np.zeros((nn_1,1))
12 Z2 = np.zeros((nn_2,1)) # Inicialización de los vectores de entrada y salida de las capas
13 A2 = np.zeros((nn_2,1))
```

Se define la topología de la red, los valores de los sesgos y pesos, de manera arbitraria; y los vectores de propagación de la red, como numéricos.

```
1 def propaga(Xo): # Función de propagación hacia adelante
2     Z1 = np.dot(W1,Xo) + b1 # Z1 = W1.Xo + W10
3     A1 = sig(Z1) # A1 = F(Z1)
4     Z2 = np.dot(W2,A1) + b2 # Z2 = W2.A1 + W20
5     A2 = sig(Z2) # A2 = F(Z2)
6     #print("A2 =", A2)
7     return A2, Z2, A1, Z1
8
9 def sig(s): # Funcion de activacion sigmoide
10     return 1/(1+np.exp(-s))
11
12 def dSig(s):
13     df = sig(s.T[0])*(sig(s.T[0]))
14     ds = np.diag(df)
15     return ds
```

Se define la función de propagación, su parámetro es solo el vector de datos de entrada X_o , y hace la propagación a través de la red, con base en los vectores de sesgo y las matrices de pesos. Encuentra los vectores de resultados en cada capa: Z_1 , A_1 , Z_2 y A_2 . Luego viene la función sigma. Y de la derivada de sigma, para el algoritmo de Backpropagation.

```

1 X = np.array([[0.05], [0.10]])
2 Yr = np.array([[0.01], [0.99]])

```

X es el vector de entrada a la red o vector de características, con los valores 0.05 Y 0.10. Luego el vector Yr, que es el resultado que queremos que aprenda, es el vector de rótulos.

```

1 A2, Z2, A1, Z1 = propaga(X)
2 print(A2)

```

```

[[0.69051105]
 [0.72421384]]

```

Con base en el vector de características X, se encuentran las salidas de cada capa de la red, no interesa la última A2, para contrastarla contra el vector de rótulos Yr.

```

1 def error(A2, Yd):
2     err = Yd - A2
3     return err

```

Permite calcular el error entre lo que produce la red, A2 y lo que se espera que debe dar, Yr.

```

1 def backpropagation(X,Yr,A2, Z2, A1, Z1): # Retropropagación
2     delta2 = np.dot(dSig(Z2),error(A2,Yr))
3     dEdW2 = -np.dot(delta2,A1.T)           # componente de corrección para W2
4     dEdb2 = -delta2
5
6     delta1 = np.dot(np.dot(dSig(Z1),W2.T),delta2)
7     dEdW1 = -np.dot(delta1,X.T)           # componente de corrección para W1
8
9     dEdb1 = -delta1
10    return dEdW2,dEdb2,dEdW1,dEdb1

```

El algoritmo de retropropagación, funcional, cuyas entradas son las salidas de cada una de las capas, el vector de entrada X y el rótulo Yr.

```

1 dEdW2,dEdb2,dEdW1,dEdb1 = backpropagation(X,Yr, A2, Z2, A1, Z1)

```

Se llama la función de retropropagación y produce los cambios que se deben hacer a los vectores de sesgo y a las matrices de pesos.

```

1 eta = 0.1

```

Definición de la tasa de aprendizaje

```

1 W2 = W2 - eta * dEdW2
2 b2 = b2 - eta * dEdb2
3 W1 = W1 - eta * dEdW1
4 b1 = b1 - eta * dEdb1

```

Se actualizan los pesos, es la primera iteración o época del aprendizaje.

```

1 A2, Z2, A1, Z1 = propaga(X)
2 print(A2)

```

```

[[0.67821878]
 [0.72890846]]

```

Los valores han mejorado, aunque muy poco.

```

1 i = 0
2 while i < 50000:
3     A2, Z2, A1, Z1 = propaga(X)
4     err = error(A2, Yr)
5
6     dEdW2, dEdb2, dEdW1, dEdb1 = backpropagation(X, Yr, A2, Z2, A1, Z1)
7     W2 = W2 - eta * dEdW2
8     b2 = b2 - eta * dEdb2
9     W1 = W1 - eta * dEdW1
10    b1 = b1 - eta * dEdb1
11
12    i = i + 1
13 print(A2)

```

```

[[0.03341429]
 [0.9900029 ]]

```

Es la forma en que se corre el aprendizaje de una red neuronal aquí por 50.000 épocas. El resultado ha mejorado. Se diría que el segundo vector está bien pero falta el otro resultado. Este es el proceso de aprendizaje de una red neuronal.

Ejemplo 2. Reconocimiento de vocales. Se definen vocales de tamaño 4 X 5 pixeles. Se entrena la red y luego se le da una vocal para que se reconozca. Por favor cargar ReconocVocales y correrlo.

De números con base en el data set MNIST de números escritos a mano, hacer reconocimiento de números escritos a mano.

Uso de librerías para la construcción de modelos

Se va a hacer una introducción al uso de tensorflow, que permite la creación de modelos usando diferentes lenguajes. En el ambiente Python se instala con: `pip install tensorflow`. Veamos un ejemplo:

Se dan una serie de puntos en los arreglos **xs** y **ys**. Se busca un modelo que permita la predicción del valor **y** para cualquier nuevo punto **x**. A c

```

1 import tensorflow as tf
2 import numpy as np
3 from tensorflow.keras import Sequential
4 from tensorflow.keras.layers import Dense
5
6 l0 = Dense(units=1, input_shape=[1])
7
8 model = Sequential([l0])
9 model.compile(optimizer='sgd', loss='mean_squared_error')
10 xs = np.array([-1.0, 0.0, 1.0, 2.0, 3.0, 4.0], dtype=float)
11 ys = np.array([-3.0, -1.0, 1.0, 3.0, 5.0, 7.0], dtype=float)
12 history = model.fit(xs, ys, epochs=100)

```

Se importan las librerías requeridas, se tiene solo una neurona, con función de activación sigmoideal y una función de pérdida de mínimos cuadrados. Se entrena el modelo durante 100 épocas.

```

1 # 1. Entrenamos el modelo (aquí usamos verbose=0 para que no llene la pantalla)
2 #history = model.fit(x_train, y_train, epochs=5, verbose=0)
3
4 # 2. Extraemos la lista de pérdidas
5 lista_perdidas = history.history['loss']
6
7 # 3. Recorremos e imprimimos cada época con su respectivo error
8 print("--- RESUMEN DEL ENTRENAMIENTO ---")
9 for epoca, perdida in enumerate(lista_perdidas):
10     print(f"Época {epoca + 1}: Pérdida = {perdida:.4f}")

```

```

--- RESUMEN DEL ENTRENAMIENTO ---
Época 1: Pérdida = 6.9013
Época 2: Pérdida = 5.6192
Época 3: Pérdida = 4.6067
Época 4: Pérdida = 3.8063
Época 5: Pérdida = 3.1728
Época 6: Pérdida = 2.6707
Época 7: Pérdida = 2.2722
Época 8: Pérdida = 1.9551
Época 9: Pérdida = 1.7022
Época 10: Pérdida = 1.4998

```

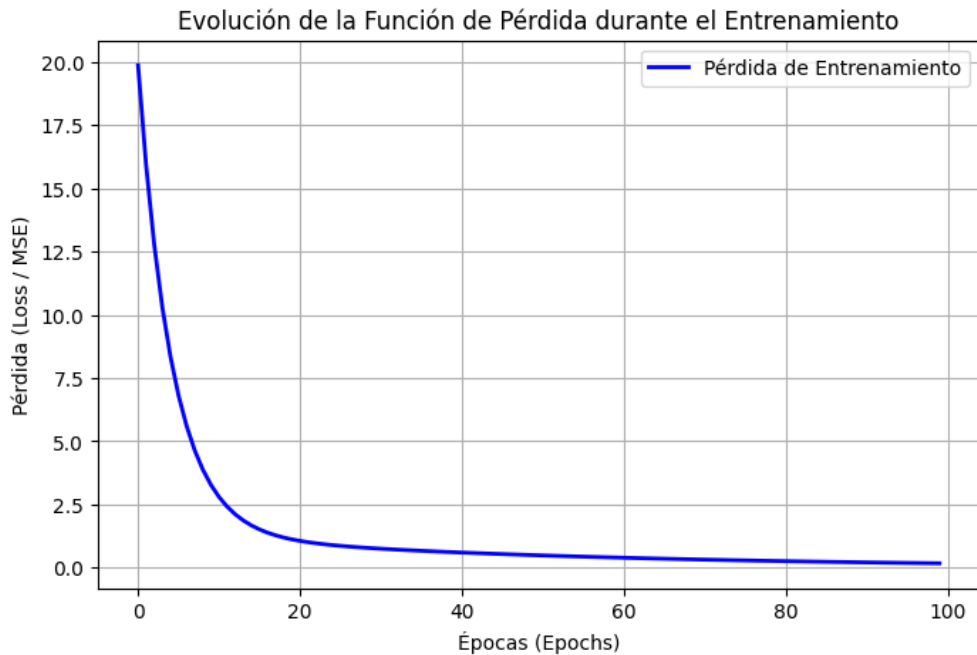
Se presentan las 10 primeras épocas, se puede observar cómo va disminuyendo la pérdida.

```

1 # Instrucciones para graficar la pérdida
2 plt.figure(figsize=(8, 5))
3 plt.plot(history.history['loss'], label='Pérdida de Entrenamiento', color='blue', linewidth=2)
4
5 # Personalización de la gráfica
6 plt.title('Evolución de la Función de Pérdida durante el Entrenamiento')
7 plt.xlabel('Épocas (Epochs)')
8 plt.ylabel('Pérdida (Loss / MSE)')
9 plt.grid(True)
10 plt.legend()
11
12 # Mostrar la gráfica en pantalla
13 plt.show()

```

Instrucciones para dibujar la función de pérdida.



5.7 Aprendizaje profundo (Deep Learning)

Como se puede apreciar en la figura 5.19, en Deep Learning se tiene una RNA, como las que se han estudiado, simplemente que el adjetivo “Deep” hace referencia a que el número de capas ocultas es relativamente alto, hablamos del orden de 7, 8 o 9 capas. Claramente funciona el mismo algoritmo de retropropagación.

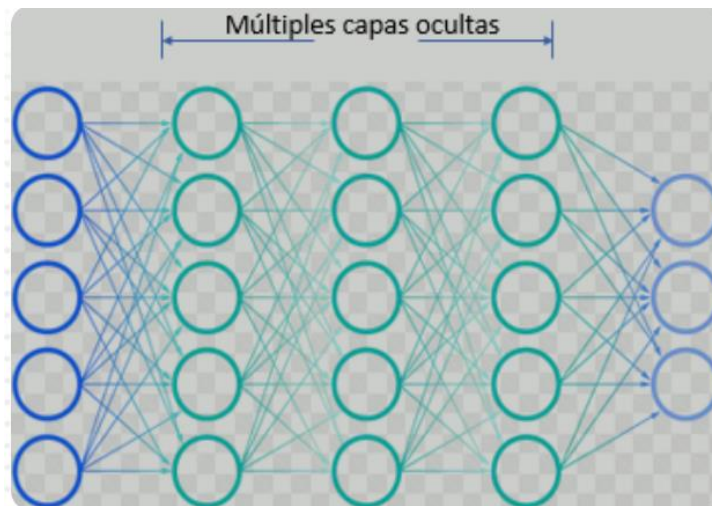


Figura 5.19. Red neuronal profunda. Con varias capas ocultas.

5.8 Hacia Deep learning, conceptos básicos de computación visual

5.8.1 Redes convolucionales.

Una convolución es simplemente un filtro de pesos que se usa para multiplicar un pixel por sus vecinos, para obtener un nuevo valor para ese pixel. En el caso del reconocimiento de

imágenes con redes neuronales se puede tener mayor claridad sobre su utilización. Veamos por ejemplo el caso en que se quiere hacer el reconocimiento de una cara. Una cara tiene una serie de características interesantes: los ojos no quedan muy distantes entre sí, bajo los ojos está centrada la nariz y bajo la nariz esta la boca. Estos elementos que conforman la cara se consideran detectores locales, figura 5.20. Los detectores locales buscan en la imagen puntos localmente interesantes.

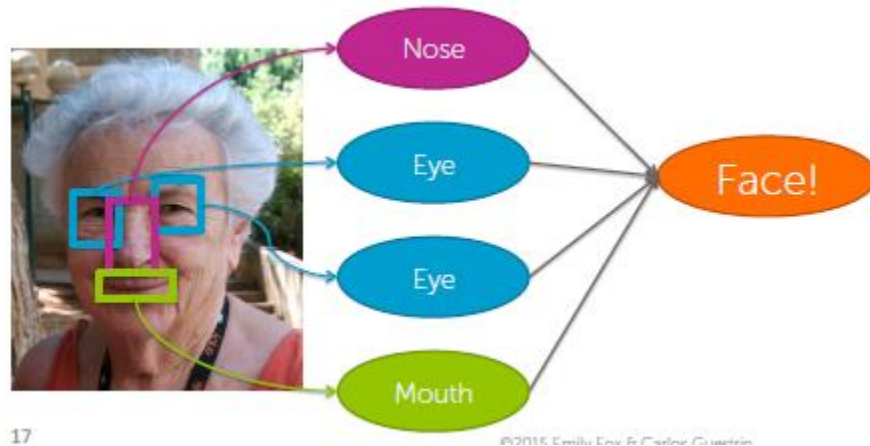


Figura 5.20. Imagen con detectores locales-

Las colecciones de puntos localmente interesantes se combinan para construir clasificadores. Con Dee Learning se aprenden implícitamente estos rasgos (features).

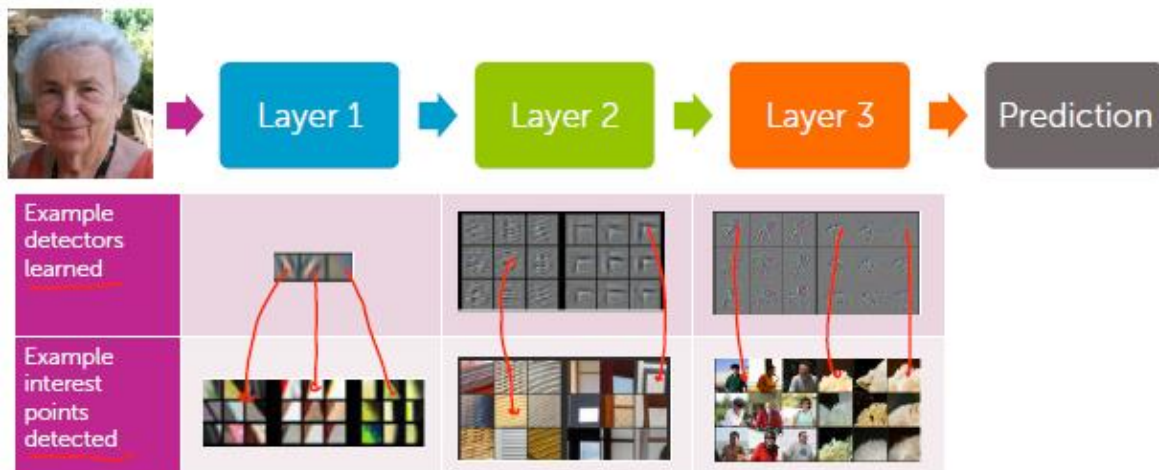


Figura 5.21. En cada capa se va aprendiendo una serie de rasgos, hasta que en la última capa se hace la predicción o reconocimiento.

Pero además se puede pensar que estos rasgos son comunes a una gran cantidad de imágenes. Luego las primeras capas lo que hacen es encontrar rasgos que son comunes a muchas imágenes. Esta es la idea de la transferencia de aprendizaje en redes con muchas capas.

En el caso de la figura 5.22, la red neuronal se conforma de 8 capas para realizar la clasificación entre perros y gatos. Una vez que la red a aprendido esta clasificación, es decir, se tiene los pesos calculados en cada capa para realizar esta clasificación, se puede tomar la misma red neuronal dejando estos pesos ya calculados hasta la capa 7, incluir otra capa para que prenda a reconocer

otra clase de imágenes, como sillas, elefantes, carros y cámaras de fotografía. Realmente es muy provechoso aprovechar el conocimiento de las capas previas.

Figura 5.21.

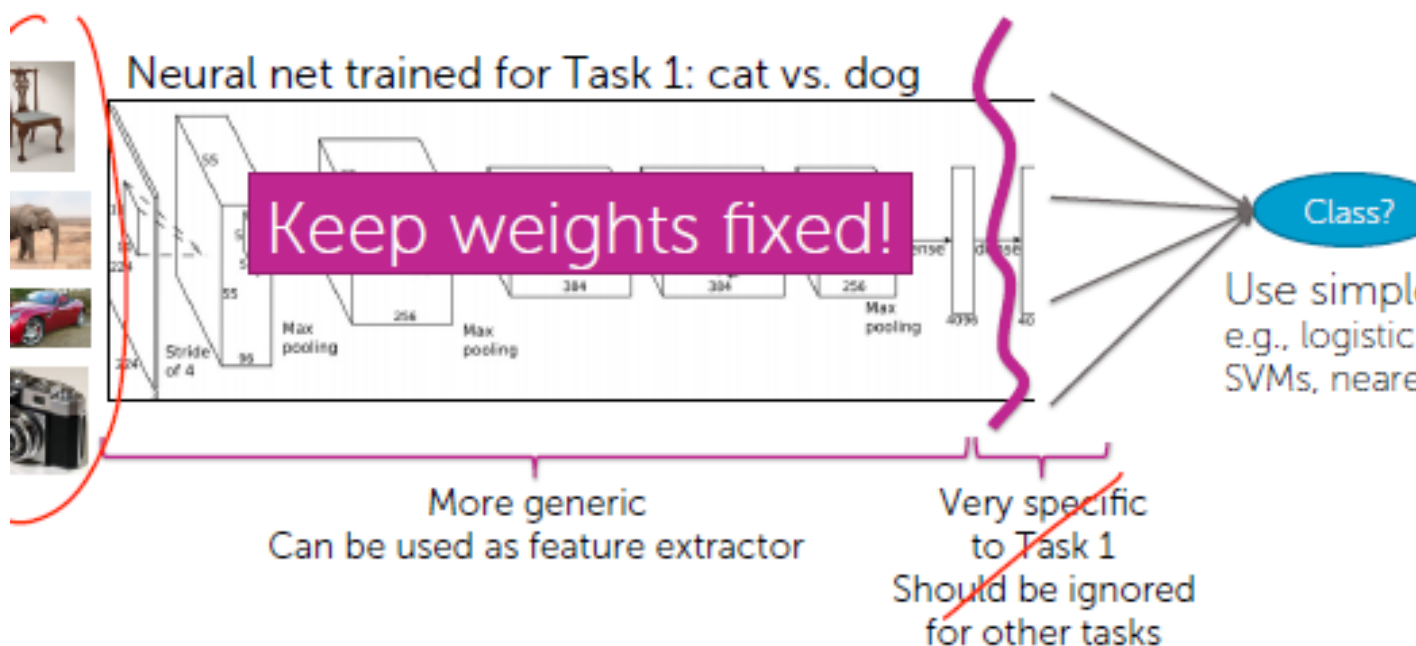


Figura 5.22. Red neuronal de ocho capas. El aprendizaje para clasificar perros y gatos se puede utilizar en otro tipo de reconocimiento o clasificación.

Veamos un ejemplo, para detectar bordes. La idea es que a la matriz que representa una imagen se le aplican ciertos filtros que permiten detectar los bordes. En este caso tenemos la matriz:

3	0	1	2	7	4
1	5	8	9	3	1
2	7	2	5	1	3
0	1	3	1	7	8
4	2	1	6	2	8
2	4	5	2	3	9

Veamos ahora en que consiste la operación convolución. Tomamos una matriz 3 X 3 y la usamos como un filtro

$$\begin{pmatrix} 3 & 0 & 1 & 2 & 7 & 4 \\ 1 & 5 & 8 & 9 & 3 & 1 \\ 2 & 7 & 2 & 5 & 1 & 3 \\ 0 & 1 & 3 & 1 & 7 & 8 \\ 4 & 2 & 1 & 6 & 2 & 8 \\ 2 & 4 & 5 & 2 & 3 & 9 \end{pmatrix} * \begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Imagen 6 X 6 Filtro 3 X 3

Para aplicar el filtro, primero la matriz de la imagen 4 X 4, se divide en 16 matrices, 3 X 3

3 0 1 1 5 8 2 7 2	0 1 2 5 8 9 7 2 5	1 2 7 8 9 3 2 5 1	2 7 4 9 3 1 5 1 3
1 5 8 2 7 2 0 1 3	5 8 9 7 2 5 1 3 1	8 9 3 2 5 1 3 1 7	9 3 1 5 1 3 1 7 8
2 7 2 0 1 3 4 2 1	7 2 5 1 3 1 2 1 6	2 5 1 3 1 7 1 6 2	5 1 3 1 7 8 6 2 8
0 1 3 4 2 1 2 4 5	1 3 1 2 1 6 4 5 2	3 1 7 1 6 2 5 2 3	1 7 8 6 2 8 2 3 9

Ahora, cada elemento de la nueva matriz se calcula tomando cada una de las submatrices y multiplicándola, de una manera especial, por el filtro.

$$\begin{bmatrix} 3 & 0 & 1 \\ 1 & 5 & 8 \\ 2 & 7 & 1 \end{bmatrix} * \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$$

Así se calcula el primer elemento de la nueva matriz:

$$3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times -1 + 8 \times -1 + 2 \times -1 = -5$$

Se hace el mismo cálculo para todas las otras submatrices, obteniéndose la siguiente matriz:

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

De esta manera al hacer la convolución de una matriz 6 X 6 con un filtro 3 X 3, obtenemos una matriz 4 X 4. De la misma manera se calculan los siguientes elementos de la matriz 4 X 4:

-5	-4	0	8
-10	-2	2	3
0	-2	-4	-7
-3	-2	-3	-16

Algunos de los problemas de visión computacional que se resuelven con CNN son: clasificación de imágenes, detección de objetos y transferencia de estilo neuronal. Un problema importante con los problemas de visión computacional es que los datos de entrada son realmente cuantiosos.

5.8.2 Polling

Pooling

Pooling es el proceso de eliminar pixels en una imagen, mientras se mantiene la semántica del contenido dentro de la imagen. Esto se explica mejor visualmente. La figura 5.23, muestra el concepto de un max pooling.

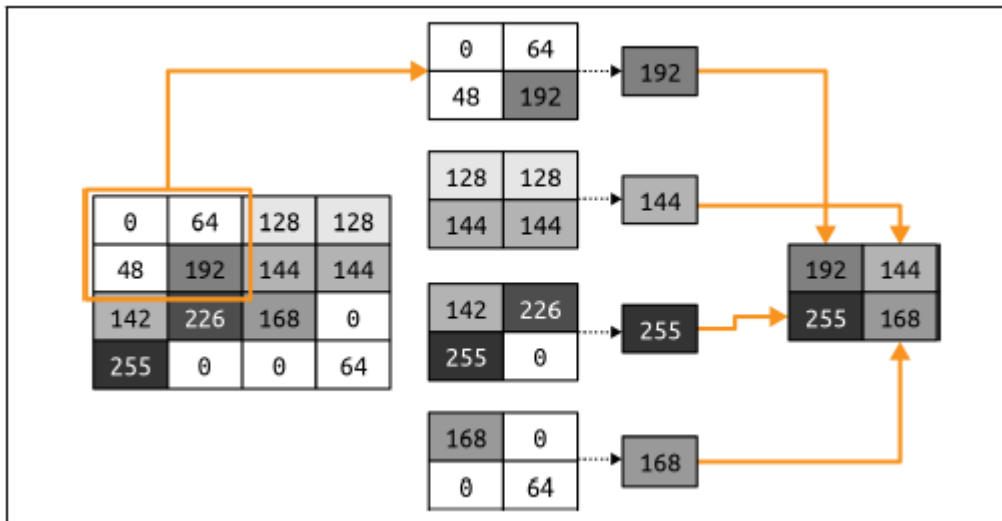


Figura 5.23. Demostración de max pooling

En este caso, considere que la caja de la izquierda son los píxeles que están una imagen monocromática. Los agrupamos en arreglos de 2 X 2, en este caso los 16 bits se agrupan en 4 arreglos de 2 X 2. Estos se llaman *pools*.

Cuando seleccionamos el máximo valor, en cada uno de los grupos, y los reagrupamos en una nueva imagen. De esta manera los pixels de la izquierda se reducen en un 75% (de 16 a 4), con el valor máximo de cada pool conformando la nueva imagen. Ascendente de la figura 5.24. Con las líneas verticales ampliadas, después de que se aplica el max pooling.

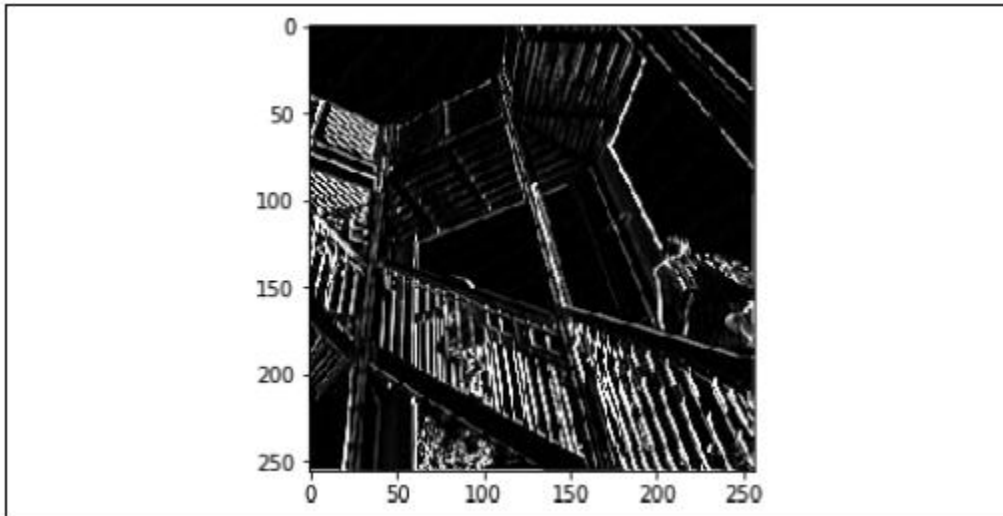


Figura 5.24. Ascendente después del filtro vertical y el max pooling

Note que las características filtradas no solo se han mantenido, sino que además han mejorado. También el tamaño de la imagen ha cambiado de 512 X 512 a 256 X 256, un cuarto del tamaño original.

Ejemplo

Se crea un modelo para reconocer imágenes de 28 X 28 pixels. Se define una red de 128 conectadas, con función de activación Relu. La escogencia de 128 capas es de alguna manera caprichosa, se diseña el número de capas para escoger el número apropiado de valores que le permitan al modelo realmente aprender, los parámetros. A veces se experimenta con diferentes redes para escoger mejor los parámetros, este proceso se llama sintonización de hiperparámetros. La última instrucción hace referencia a la capa de salida que califica las imágenes en 10 clases, Utiliza la función de activación softmax, que produce valores entre 0 y 1.

```
1 model = keras.Sequential([
2     keras.layers.Flatten(input_shape=(28, 28)),
3     keras.layers.Dense(128, activation=tf.nn.relu),
4     keras.layers.Dense(10, activation=tf.nn.softmax)
5 ])
```

5.9. Ejercicios.

1. Tome una tabla de datos de un experimento cualquiera. Encuentre un modelo con base en una red neuronal de dos capas ocultas. Dele datos a ese modelo para hacer una predicción.
2. Con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Use una red neuroanl con dos capas ocultas y pruebe el modelo, ¿qué precisión obtuvo? Saque conclusiones.

3. Con base en la librería tensorflow, descargue el data set fashion MNIST. Haga una clasificación de prendas de vestir. Explique cada una de las funciones y las principales instrucciones, saque conclusiones.
4. De la librería de Kaggle para Python, baje un dataset, estudie sus características (features) y su rótulo. Genere un modelo y tome ejemplos de internet y analice los resultados.
5. Punto libre.

5.10 Bibliografía

1. K. He, X. Zhang, S. Ren y J. Sun, Deep Residual Learning for Image Recognition, Microsoft Research.
3. Laurence Moroney, AI and Machine Learning for Coders *A Programmer's Guide to Artificial Intelligence*.
4. A. Ng, Machine Learning, Coursera